CONOCIMIENTO
COMPROMISO
COMUNIDAD

EDUCACIÓN
TECNOLOGÍA
OPORTUNIDADES

Formando futuros posibles

SISTEMA DE ASISTENCIA

Instituto Superior de Formación Técnica N.º 38



Proyecto Institucional

Francisco Herrera

IDEAS
PERSONAS
SOLUCIONES
UN MEJOR MAÑANA

Manual del Desarrollador

Sistema de Asistencia del ISFT N.° 38

Versión actual del sistema	4.0
Fecha de actualización del manual	28 de octubre de 2026
Responsable	Francisco Herrera
Repositorio	github.com/franciscoherrera-coder/proyecto
Framework	Laravel 8 / PHP

Historial de versiones del sistema

Versión	Fecha	Alcance funcional
1.0	12/07/2026	Primera etapa del módulo: vistas para preceptores y directora, control de acceso administrativo, gestión inicial de usuarios y carreras, y funciones básicas para consultar y administrar materias.
2.0	16/08/2026	Incorporación de las vistas de profesores y alumnos. Se agregaron asignaciones a materias, listados de estudiantes, perfiles, consulta de materias e historial individual.
3.0	14/09/2026	Desarrollo de la asistencia mediante código QR: generación de códigos para presente o tardanza, habilitación e inhabilitación, lectura desde el teléfono y validación del alumno y la materia.
4.0	21/10/2026	Implementación y consolidación de la planilla de asistencia: carga diaria de estados, motivos de justificación, consulta histórica, cálculo de porcentajes, impresión e integración con los demás módulos.

Propósito del documento

Permitir que otro desarrollador comprenda, instale, pruebe, mantenga y amplíe el módulo sin depender del autor original. Las afirmaciones se basan en el código fuente del repositorio y deben validarse nuevamente contra el entorno productivo.

Índice

1. Definición general	2. Requisitos y límites
3. Entorno y tecnologías	4. Arquitectura y módulos
5. Modelo de datos	6. Procesos principales
7. Rutas, métodos y código	8. Instalación y despliegue
9. Pruebas	10. Seguridad
11. Mantenimiento	12. Conclusiones y anexos

1. Definición general

El módulo integra en el portal Laravel del ISFT N.° 38 la gestión de usuarios, carreras, materias, asignaciones, planillas diarias, códigos QR e indicadores. Se desarrolló para centralizar registros, reducir duplicados y aplicar permisos por rol y ámbito académico.

¿Para qué funciona la sección de asistencia?

La sección de asistencia funciona como el espacio central para registrar, organizar y consultar la presencia de los alumnos en cada materia y fecha de cursada. Permite reemplazar las planillas manuales por registros digitales relacionados con los alumnos, profesores, materias, carreras y horarios existentes en la plataforma.

Desde esta sección, los preceptores y la directora configuran usuarios, materias y asignaciones; los profesores completan la planilla diaria o habilitan un código QR; y los alumnos registran su asistencia mediante ese código y consultan su historial. A partir de los estados guardados —presente, ausente, tarde o justificado— el sistema calcula porcentajes y facilita la identificación de alumnos que se encuentran por debajo del mínimo requerido.

Su finalidad es proporcionar información centralizada, actualizada y accesible según el rol de cada usuario, agilizar la toma de asistencia, reducir errores y duplicaciones, y facilitar el seguimiento académico y la generación de listados para consulta o impresión.

Usuarios y audiencia técnica

Usuarios funcionales: directora de asistencia, preceptores, profesores y alumnos. Audiencia de este manual: desarrolladores PHP/Laravel, responsables de MySQL/MariaDB, despliegue y soporte, con experiencia intermedia en MVC, SQL, Git y línea de comandos.

Objetivos

- Registrar un estado único por alumno, materia y fecha.
- Ofrecer carga manual y QR revocable para presente/tardanza.
- Calcular historial y porcentaje con reglas institucionales.
- Limitar preceptores por carrera y profesores por materia.
- Reutilizar entidades del portal sin romper compatibilidad heredada.

Alcance

Incluye el submódulo web bajo **/asistencia**. Reutiliza `users`, `registros`, `carreras`, `materias`, `profesors`, `horarios` y `anios`, e incorpora tablas de asistencia, QR y pivotes. No incluye API pública, aplicación móvil nativa, biometría, carga de certificados ni edición histórica desde interfaz.

2. Requisitos

2.1 Requisitos funcionales

ID	Descripción verificable
RF-01	Autenticar por DNI o correo y contraseña; regenerar sesión y cerrar de forma segura.
RF-02	Resolver roles: alumno (0/nulo), administrador/preceptor (1) y profesor (2).
RF-03	Limitar al preceptor a sus carreras y al profesor a sus materias.
RF-04	Registrar alumnos, validar profesores y crear/asignar preceptores.
RF-05	Vincular cuenta y ficha académica por DNI.
RF-06	Asignar alumno–materia y profesor–horario/materia.
RF-07	Guardar estado único por alumno/materia/fecha y editar solo hoy en día de cursada.
RF-08	Generar token QR aleatorio, revocar anteriores y admitir solo alumno asignado.
RF-09	Calcular tardanza como 0,5 y justificada como 0; mostrar historial.
RF-10	Alertar bajo 80 % en prácticas y 70 % en otras materias.
RF-11	Filtrar porcentajes e imprimir planillas/listados.
RF-12	Editar configuración de carrera y materia desde el ámbito administrativo.

2.2 Requisitos no funcionales

- Aplicación responsive en español y navegadores actuales.
- Integridad mediante claves foráneas, unicidad y transacciones.
- Hash de contraseñas, CSRF, cookies/sesión y autorización de ámbito.
- MVC con Blade/Eloquent y migraciones reproducibles.
- Configuración y secretos fuera del repositorio mediante `.env`.

2.3 Restricciones

El módulo depende de un esquema académico heredado y de horarios configurados. Conserva relaciones indirectas por DNI y, en profesores, por identidad asociada al horario. La directora se identifica actualmente mediante un correo técnico fijo.

3. Entorno y tecnologías

Capa	Tecnología	Uso
Servidor	PHP ^7.3 | ^8.0; Laravel ^8.75	Routing, sesión, validación, MVC, Eloquent y migraciones.
Datos	MySQL/MariaDB, InnoDB, utf8mb4	Persistencia relacional e integridad.
Vista	Blade, Bootstrap 4.6, jQuery 3.6, CSS/JS	Renderizado del servidor e interfaz responsive.
Build	Laravel Mix 6, Sass, npm	Compilación de activos.
Front externo	Flatpickr, QRCode.js, Html5Qrcode	Fecha, representación QR y cámara.
Documentos	mPDF 8.2, Dompdf 2.1	Disponibles en Composer; la UI usa impresión del navegador.
Control	Git	Versionado y ramas.

Decisiones principales

- **Monolito y Blade:** integra rápido con el portal y reutiliza su sesión/modelos.
- **Controlador concentrado:** simplificó la incorporación inicial, a costa de tamaño y acoplamiento.
- **QR persistido:** permite habilitar/inhabilitar, a diferencia de un enlace únicamente firmado.
- **`updateOrCreate` + índice único:** hace idempotentes planilla y escaneo.
- **Renderizado del servidor:** evita mantener una API/SPA adicional.

> Sanctum está instalado en el proyecto, pero este módulo utiliza autenticación web de sesión.

4. Arquitectura

Flujo general de una solicitud

1. Navegador	El usuario utiliza las vistas Blade mediante una conexión HTTP. La sesión identifica su rol y los formularios incluyen protección CSRF.
↓	
2. Rutas	`routes/asistencia.php` recibe la solicitud y la dirige al método correspondiente del controlador.
↓	
3. Controlador	`AsistenciaController` autentica, autoriza, valida los datos y ejecuta el proceso solicitado.
↓	
4. Modelos y base	Eloquent consulta o modifica la información almacenada en MySQL/MariaDB, usando transacciones cuando la operación afecta varios registros.
↓	
5. Respuesta	El sistema devuelve una vista Blade o una redirección con un mensaje de confirmación o error.

Estructura de módulos funcionales

N.º	Módulo	Tipo
1	**Preceptores y directora**	Vistas administrativas
2	**Profesores y alumnos**	Vistas académicas
3	**Asistencia mediante QR**	Registro rápido de presente o tardanza
4	**Planilla de asistencia**	Registro y consulta diaria

La división anterior representa las funciones que percibe el usuario y permite localizar cada responsabilidad sin confundirla con las capas técnicas de Laravel. Los cuatro módulos comparten rutas, autorización, modelos Eloquent y el controlador del sistema.

Módulos y responsabilidades

1. Preceptores y directora — vistas administrativas	
Responsabilidad	Administrar alumnos, profesores, preceptores, carreras, materias y asignaciones. La directora accede a todas las carreras; cada preceptor solo a las asignadas.
Necesita	Sesión administrativa, relación `carrera_user`, catálogos académicos y validación de ámbito.
Implementación	`index.blade.php`, `materia-admin.blade.php`, `materia-admin-con-perfiles.blade.php`, rutas `/asistencia/admin/*` y métodos administrativos de `AsistenciaController`.

2. Profesores y alumnos — vistas académicas	
Responsabilidad	Mostrar materias, alumnos, historial, porcentajes, perfiles y listados, diferenciando las acciones del profesor de las consultas del alumno.
Necesita	Usuario autenticado, horarios, profesor asociado, vínculo `materia_registro` y registros de asistencia.
Implementación	`materia-profesor.blade.php`, `materia-alumno.blade.php`, vistas de porcentajes, perfil y listado del profesor.

3. Asistencia mediante QR	
Responsabilidad	Generar un QR para presente o tardanza, inhabilitar códigos anteriores y registrar al alumno sin crear duplicados.
Necesita	Materia y fecha actual, profesor autorizado, alumno asignado, token aleatorio, cámara o navegador y tabla `codigos_qr_asistencia`.
Implementación	Rutas de generación, cambio de estado y lectura del QR; métodos `generarQrAsistencia`, `cambiarEstadoQrAsistencia` y `registrarAsistenciaPorQr`.

4. Planilla de asistencia	
Responsabilidad	Registrar presente, ausente, tarde o justificado; consultar fechas anteriores e imprimir. Conserva un único estado por alumno, materia y fecha.
Necesita	Horario con día configurado, profesor asignado, alumnos de la materia, validación de fecha y motivo, y tabla `asistencias_diarias`.
Implementación	`planilla-diaria-profesor.blade.php`, rutas GET/POST de planilla y métodos `planillaDiariaProfesor` y `guardarPlanillaDiariaProfesor`.

Dependencias funcionales

Una materia aparece por su horario; el profesor se vincula mediante horario; un alumno por `materia_registro`; el preceptor por `carrera_user`. Las guardas del controlador deben mantenerse porque login, registro y consumo inicial del QR no están globalmente detrás de `auth`.

5. Modelo de datos

Entidades propias

Tabla	Campos / restricciones relevantes
`asistencias_diarias`	`materia_id`, `registro_id`, fecha, estado, motivo; FKs en cascada; único (materia, registro, fecha); estados de aplicación: presente, ausente, tarde, justificado.
`codigos_qr_asistencia`	Materia, creador opcional, fecha, token único de 64 caracteres, tipo presente/tarde, habilitado e índice materia/fecha.
`materia_registro`	Pivote único alumno–materia con timestamps.
`carrera_user`	Pivote preceptor–carrera; la sincronización mantiene un responsable activo.
`carrera_profesor_user`	Pivote de profesor validado por carrera.

Entidades reutilizadas

`users` guarda acceso, DNI e `is_admin`; `registros`, la ficha académica; `materias`, `horarios`, `carreras`, `anios` y `profesors` aportan el catálogo académico. En horarios, el día usa ISO 1=lunes ... 7=domingo.

Clasificación de datos

- **Entrada:** formularios, fecha, estados y token escaneado.
- **Internos:** roles, asignaciones, tokens, permisos y estados.
- **Salida:** vistas, mensajes, indicadores e impresión del navegador.

6. Procesos principales

6.1 Autenticación

```
validar identificador y contraseña si identificador es numérico y users.dni existe: buscar por dni
si no: buscar por email Auth::attempt → regenerar sesión → redirigir al panel
```

6.2 Guardado de planilla

Entrada: fecha y `asistencias[registro_id][estado,motivo]`. Verifica profesor/materia, fecha no futura, día permitido, fecha igual a hoy, motivo obligatorio para justificado y pertenencia del alumno. Dentro de una transacción ejecuta `updateOrCreate`. **Salida:** redirección con mensaje.

6.3 Registro por QR

Busca token, exige habilitado y fecha de hoy; si no hay sesión envía al login; luego exige rol alumno y asignación. Crea o actualiza con el tipo del QR. La clave única impide duplicados.

6.4 Cálculo

```
clases_totales = fechas con planilla en la materia presencias = presentes + tardanzas × 0,5
porcentaje = presencias / clases_totales × 100 sin fila individual en una fecha con clase =
ausencia automática justificada = se informa, pero suma 0
```

6.5 Administración

La directora sincroniza preceptores/carreras en transacción y confirma reemplazos. El preceptor opera en `carreraIdsUsuario`. Un profesor debe estar validado en la carrera antes de recibir materias.

7. Rutas y métodos

Todas son rutas web con HTML/redirección, sesión y CSRF; no existe API REST pública.

7.1 Acceso y administración

Verbo y ruta	Método	Entrada / propósito
GET /asistencia	index	Panel o acceso; `carrera_id?`.
POST /asistencia/login	login	`identificador, password, remember?`.
POST /asistencia/registro	registrar	Alta de alumno con datos personales y contraseña confirmada.
POST /asistencia/logout	logout	Invalida sesión y renueva token.
POST /asistencia/admin/usuarios	registrarDesdeAdmin	Directora crea preceptor con `carrera_ids[]`.
PUT .../preceptores/{user}/carreras	actualizarCarrerasPreceptor	Sincroniza carreras no vacías.
POST /admin/alumnos	crearAlumno	Crea ficha y cuenta vinculadas.
PUT /admin/alumnos/{registro}	actualizarAlumno	Edita; contraseña opcional.
DELETE /admin/alumnos/{registro}	eliminarAlumno	Eliminación y cascadas; exige ámbito.
POST /admin/usuarios/profesor	validarProfesor	`user_ids[], carrera_id`.
POST /admin/profesores/materias	asignarMateriasProfesor	`user_id, materia_ids[]`.
POST /admin/profesor	asignarProfesor	Actualiza profesor del horario.
POST|DELETE /admin/alumno	asignarAlumno / quitarAlumno	Modifica pivote materia–registro.
PUT /admin/carreras/{carrera}	actualizarCarrera	Edita configuración autorizada.
PUT /admin/materias/{materia}	actualizarMateria	Edita materia, año, profesor u orden.

7. Rutas de profesor, alumno y QR

Verbo y ruta (prefijo /asistencia)	Método / respuesta
GET /profesor/materias/{materia}	`verMateriaProfesor`; 403 sin asignación.
POST /profesor/alumnos	`asignarAlumnosProfesor`; exige misma carrera.
DELETE /profesor/materias/{materia}/alumnos/{registro}	`quitarAlumnoProfesor`; 404 si no existe.
GET .../listado	`listadoMateriaProfesor`; vista imprimible.
GET .../porcentajes[/{registro}]	Resumen o detalle de asistencia.
GET|POST .../planilla	Lee o guarda fecha y mapa de asistencias; 403/422.
POST .../qr	`generarQrAsistencia`; fecha y tipo; solo hoy.
PATCH .../qr/{codigoQr}	`cambiarEstadoQrAsistencia`; alterna habilitación.
GET /profesor/alumnos/{registro}	Perfil limitado a alumnos relacionados.
GET /alumno/materias/{materia}	Historial; 403 sin asignación.
GET /qr/{token}	Registra; 404 inexistente, 403 cerrado/rol/asignación.

Contrato de validación importante

Alumno: nombre/apellido hasta 100, email válido, DNI/CUIL enteros únicos a nivel de aplicación, carrera existente y contraseña mínima de ocho caracteres confirmada. Planilla: fecha válida, estados admitidos y motivo solo/obligatorio para justificado.

Convenciones de respuesta

- **302:** redirección después de operación o autenticación.
- **403:** usuario autenticado sin permiso o ámbito.
- **404:** recurso/token/asignación no hallados.
- **422:** validación de formulario o regla de negocio.

> Inventario ejecutable: `php artisan route:list --path=asistencia`. No hay OpenAPI porque el módulo no expone API.

7. Código y estructura

Archivos clave

Ruta	Responsabilidad
routes/asistencia.php	Contrato HTTP.
app/Http/Controllers/AsistenciaController.php	Casos de uso y guardas.
app/Models/AsistenciaDiaria.php	Estado individual diario.
app/Models/CodigoQrAsistencia.php	Token diario habilitable.
resources/views/frontend/asistencia/	Diez vistas Blade del módulo.
database/migrations/2026_08_26_*	Asistencias y motivo.
database/migrations/2026_09_03_*	Códigos QR.
database/migrations/2026_09_17_* y 2026_09_24_*	Pivotes de ámbito.

Reglas al modificar

- No eliminar guardas de rol/ámbito al extraer métodos del controlador.
- Mantener la unicidad alumno–materia–fecha y transacciones compuestas.
- No calcular/guardar porcentajes: se derivan de los registros fuente.
- Documentar parámetros, obligatoriedad, salida y errores con PHPDoc cuando se extraigan servicios.
- Agregar una prueba de regresión por cada regla cambiada.

Deuda técnica conocida

El controlador tiene responsabilidades amplias; hay comprobaciones condicionales de esquema, identidad de directora por correo, vínculo usuario–registro por DNI y asociaciones heredadas de profesor. Estos puntos requieren refactor gradual con datos de prueba.

8. Instalación y ejecución

8.1 Requisitos

PHP 8.x recomendado con extensiones requeridas por Laravel/mPDF, Composer 2, MySQL/MariaDB, Node/npm compatible con Mix 6, Git y servidor web con DocumentRoot en `public/`.

8.2 Instalación limpia

```
git clone https://github.com/franciscoherrera-coder/proyecto.git cd proyecto composer install copy .env.example .env php artisan key:generate # configurar APP_URL y DB_* en .env php artisan migrate npm install npm run production php artisan storage:link php artisan config:cache php artisan route:cache php artisan view:cache
```

En Linux/macOS reemplace `copy` por `cp`. En Laragon seleccione PHP 8.x, cree la base con utf8mb4 y apunte el host a `public/`. Desarrollo: `php artisan serve` y `npm run dev`.

8.3 Variables principales

`APP_NAME`, `APP_ENV`, `APP_KEY`, `APP_DEBUG`, `APP_URL`, `DB_CONNECTION`, `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`, además de sesión, caché, cola y correo. En producción: `APP_DEBUG=false`, HTTPS y secretos fuera de Git.

8.4 Verificación posterior

1. Abra `/asistencia`.
2. Ejecute `php artisan route:list --path=asistencia`.
3. Compruebe migraciones y permisos de `storage/`.
4. Pruebe cada rol con datos no productivos.
5. Revise `storage/logs/laravel.log` sin exponer datos sensibles.

8. Despliegue y operación

Lista de despliegue

- Respaldo probado y ventana de mantenimiento definida.
- `.env` productivo no versionado; HTTPS; debug desactivado.
- `composer install --no-dev --optimize-autoloader` y activos de producción.
- Migraciones revisadas y aplicadas con `php artisan migrate --force`.
- Cachés reconstruidas y permisos mínimos en storage/cache.
- Directora, preceptores, carreras, horarios, profesores y alumnos verificados.
- Prueba de planilla, QR, porcentaje, impresión y cierre de sesión.

Respaldo y recuperación

Antes de migrar o corregir datos: detener escrituras, generar dump cifrado, registrar commit y migración, y verificar restauración en un entorno separado. No edite el porcentaje: rectifique `asistencias_diarias` bajo procedimiento auditado.

Observabilidad

Monitorear errores 4xx/5xx, excepciones Laravel, fallas de DB, crecimiento de tablas, caducidad TLS y disponibilidad de dependencias CDN. No registrar contraseñas, tokens QR completos, DNI/CUIL ni volcados de formularios.

Compatibilidad

El `composer.json` declara PHP `^7.3|^8.0`, pero se recomienda validar con la versión exacta de producción y las extensiones instaladas. La cámara requiere contexto seguro (HTTPS) en navegadores móviles modernos.

9. Pruebas

Los archivos `tests/Feature/ExampleTest.php` y `tests/Unit/ExampleTest.php` son plantillas; no existe cobertura automática específica suficiente. Comandos:

```
php artisan test # alternativa vendor/bin/phpunit
```

Matriz mínima de aceptación

Caso	Entrada	Resultado
Login válido/inválido	DNI/correo + clave	Panel o error de credencial.
Ámbito preceptor	Carrera propia/ajena	Permitido / 403.
Planilla hoy	Cuatro estados	Una fila única por alumno/fecha.
Fecha histórica	POST anterior	422; no modifica.
Justificado sin motivo	Estado justificado	422.
QR válido y repetido	Alumno asignado	Crea/actualiza sin duplicar.
QR cerrado/antiguo	Token existente	403.
Porcentaje	2 presentes, 1 tarde, 1 ausente	62,5 %.
Sustituir preceptor	Carrera ocupada	Reemplazo transaccional confirmado.

Pruebas prioritarias a automatizar

Autorización por rol/ámbito, concurrencia QR, integridad de borrados, umbrales, motivos de justificación, fechas/zonas horarias, vínculos de identidad y compatibilidad con el esquema heredado. Incorporar CI antes de ampliar funcionalidades.

10. Seguridad y datos

- Laravel aporta cookies cifradas, sesión, CSRF y route-model binding.
- Las contraseñas usan `Hash::make`; nunca deben copiarse ni registrarse.
- El QR usa `Str::random(64)`, token único, revocable y diario.
- Cada operación sensible conserva una guarda explícita de rol y ámbito.
- Datos personales: nombre, DNI, CUIL, correo, carrera y ficha académica; aplicar minimización, HTTPS, respaldo cifrado, retención institucional y control de exportación.

> **Riesgo conocido:** la directora se reconoce por `admin.asistencia@isft38.test`. Migrar a roles/permisos persistidos antes de múltiples directoras o cambios de correo.

No versionar `.env`, logs, credenciales ni volcados reales. `dev338.sql` debe considerarse sensible y anonimizarse antes de compartir.

Amenazas y controles

Riesgo	Control actual	Mejora
Acceso cruzado	Guardas en controlador.	Policies/Gates y pruebas.
Reuso de QR	Fecha, habilitación y asignación.	Caducidad horaria/auditoría.
Duplicados	Índice + updateOrCreate.	Prueba de concurrencia.
Dependencia CDN	Navegador.	Activos locales, SRI y CSP.
Borrado irreversible	Confirmación UI y cascadas.	Soft delete/auditoría/retención.

11. Mantenimiento y evolución

Prioridades

1. Dividir `AsistenciaController` en Form Requests, Policies y servicios.
2. Agregar tests Feature/Unit y CI.
3. Reemplazar el correo especial por roles/permisos.
4. Unificar roles heredados de otros módulos.
5. Crear rectificación histórica auditada.
6. Servir dependencias del front localmente o aplicar SRI/CSP.
7. Auditar altas, bajas, cambios y QR.
8. Normalizar email/DNI/CUIL con índices DB.
9. Usar FK explícita para identidad de profesor.
10. Revisar middlewares globales de `routes/web.php`.

Procedimiento de cambio

1. Crear rama y describir regla afectada.
2. Preparar datos anonimizados y respaldo.
3. Agregar prueba que falle antes del cambio.
4. Modificar la capa mínima necesaria.
5. Ejecutar pruebas, inspección de rutas y migración de ensayo.
6. Revisar autorización, datos personales y rollback.
7. Actualizar manuales y control de versiones.

Zonas frágiles

`Schema::hasTable/hasColumn`, vínculo por DNI, coincidencias heredadas de profesor, horarios sin día, reemplazo de preceptor y cascadas. No modificar sin respaldo y casos de prueba.

12. Conclusiones y anexos

Conclusiones

El reto principal fue integrar asistencia a un esquema académico heredado con convenciones de rol y relaciones indirectas. Se adoptaron transacciones, pivotes, validación de ámbito y registros idempotentes. Se limitaron API, app móvil, certificados y edición histórica para concentrar la entrega en el circuito diario. La evolución debe hacer explícitos roles e identidades y acompañar cada refactor con pruebas.

Checklist de entrega técnica

- Secretos fuera de Git, HTTPS y debug desactivado.
- Migraciones aplicadas; respaldo y restauración comprobados.
- Roles, carreras y horarios revisados.
- Planilla, QR, porcentaje e impresión aceptados por cada rol.
- Canal de soporte y URL productiva comunicados.
- Repositorio etiquetado y documentación versionada.

Referencias

- Código, migraciones, vistas, `composer.json` y `package.json` del proyecto.
- Documentación oficial de Laravel 8: routing, authentication, validation, Eloquent, migrations y CSRF.
- Documentación de Bootstrap 4.6, Flatpickr, QRCode.js y Html5Qrcode.
- Guía suministrada “Guía para elaborar un Manual de Desarrollador”.